BÁO CÁO THỰC TẬP NGHIÊN CỨU TẠI VIETTEL

Lớp Thực Thi Tool Đáng Tin Cậy Cho LLM Agent

Reliable Tool Execution Harness trên τ^2-bench Retail

Sinh viên thực hiện:	Bùi Vũ Nhật Minh
Người hướng dẫn trực tiếp:	Nguyễn Duy Anh
Cố vấn kỹ thuật:	Nguyễn Duy Hưng – Kiến trúc sư OSS
Đơn vị:	Tổng Công ty Mạng lưới Viettel
Repository:	https://github.com/muoimeo/tau2-tool-execution-harness

Tháng 9 năm 2026

Tóm tắt

Tác tử dựa trên mô hình ngôn ngữ lớn (LLM) có thể đọc yêu cầu, gọi công cụ và thay đổi trạng thái môi trường. Một lời gọi không hợp lệ, một phản hồi provider không sử dụng được hoặc một thao tác ghi bị thực thi lặp lại đều có thể làm hỏng quá trình này. Báo cáo xây dựng một *harness* tự chọn bật trên τ^2-bench v0.1.3: **A1 kiểm tra và chặn lời gọi trước thực thi**; **A2 thử lại có giới hạn ở biên gọi provider**; **A3 sửa một số lỗi biểu diễn không mất thông tin, kiểm tra kết quả công cụ và hậu điều kiện WRITE, rồi cách ly khi không chứng minh được an toàn**. Công cụ Retail, dữ liệu nhiệm vụ, bộ đánh giá và cách tính reward của benchmark không bị thay đổi.

Thí nghiệm tự nhiên dùng cùng 58 nhiệm vụ Retail, bốn trial, tổng cộng 232 episode dự kiến cho mỗi cấu hình. Số episode có kết quả hợp lệ là 232 (A0), 232 (A1), 227 (A2) và 221 (A3). Pass[1] trên episode hoàn tất lần lượt là 80,17%, 82,76%, 83,26% và 80,09%; chênh lệch này *không* chứng minh harness cải thiện khả năng suy luận của mô hình. Trong các trace A1–A3, không quan sát thấy lời gọi bị REJECT vẫn được dispatch (0/3, 0/4 và 0/5), hoặc một WRITE cùng định danh logic được thực thi hai lần. Thử nghiệm chèn lỗi có kiểm soát trên provider cục bộ bổ sung bằng chứng cơ chế về chặn lỗi, thử lại và cách ly sau WRITE. Những lỗi vận hành không tạo kết quả benchmark được **thống kê riêng, không bị chuyển thành reward bằng 0**.

Từ khóa: tác tử LLM; gọi công cụ; kiểm tra trước thực thi; thử lại provider; hậu điều kiện WRITE; chèn lỗi có kiểm soát.

Mục lục

Tóm tắt **1**

1 Giới thiệu **5**
1.1 Bối cảnh và vấn đề 5
1.2 Câu hỏi nghiên cứu và đóng góp 5

2 Nền tảng và phạm vi lỗi **6**
2.1 Luồng thực thi gốc của τ^2-bench 6
2.2 Phân loại để tránh quy lỗi nhầm 6

3 Thiết kế và triển khai harness **7**
3.1 Nguyên tắc tích hợp 7
3.2 A0: baseline native 7
3.3 A1: kiểm tra và thực thi an toàn 7
3.4 A2: thử lại ở biên provider 8
3.5 A3: sửa biểu diễn có giới hạn, kiểm tra đầu ra và cách ly 8
3.6 Artifact và tính kiểm toán 9

4 Phương pháp thực nghiệm **10**
4.1 Benchmark tự nhiên và cấu hình đóng băng 10
4.2 Định nghĩa metric và mẫu số 10
4.3 Chèn lỗi có kiểm soát 11

5 Kết quả và phân tích **12**
5.1 Kết quả benchmark tự nhiên 12
5.2 An toàn thực thi và chi phí vận hành 12
5.3 Bằng chứng riêng của A3 13
5.4 Kết quả chèn lỗi và phát biểu tăng thêm 13
5.5 Phân tích reward 0 và lỗi vận hành 13

6 Giới hạn, khả năng tái lập và hướng tiếp theo **15**
6.1 Giới hạn suy luận từ kết quả 15
6.1.1 Benchmark tự nhiên chưa tạo đủ áp lực cho lớp thực thi 15
6.1.2 Fault injection có kiểm soát chưa phản ánh đầy đủ lỗi thực tế 15
6.1.3 Bộ metric chưa bao quát toàn bộ độ tin cậy vận hành 15

6.1.4 A3 là phục hồi có kiểm soát, chưa phải phục hồi tự chủ 16
6.2 Hướng phát triển có điều kiện 16
6.2.1 Đánh giá nhiều mô hình và nhiều mức năng lực 16
6.2.2 Mô hình hóa fault sát với run tự nhiên hơn 16
6.2.3 Tiến từ containment tới phục hồi tự động có chứng minh 16
6.3 Khả năng tái lập và đối chiếu artifact 17

7 Kết luận **18**

A Nguồn số liệu và phiên bản run **19**

B Định nghĩa đối chiếu nhanh các trường **20**

Danh sách bảng

2.1 Các nguồn lỗi có mẫu số và trách nhiệm khác nhau. 6

3.1 Vị trí và giới hạn của từng tầng harness. 9

5.1 Kết quả trên các run tự nhiên cuối hợp lệ; Pass[1], Pass[4] và DB match dùng mẫu số episode/task hoàn tất tương ứng. 12
5.2 Quan sát thực thi trên run tự nhiên. A0 không có A1 trace để enforce các bất biến tương ứng; — không có nghĩa bằng 0. 13
5.3 Tóm tắt FI theo khả năng thực thi, không phải reward benchmark. 14
5.4 Phân loại nguyên nhân chính của episode đã hoàn tất nhưng reward 0; không bao gồm operationally unresolved. 14

Chương 1

Giới thiệu

1.1 Bối cảnh và vấn đề

Trong một tác tử dùng công cụ, đầu ra của LLM không chỉ là câu trả lời cho người dùng. Nó có thể chứa tên công cụ và bộ đối số mà môi trường sẽ thực thi. Một tác vụ Retail thường gồm nhiều lượt hội thoại và nhiều lệnh READ/WRITE. Lỗi định dạng hoặc chọn sai công cụ có thể xuất hiện ở bất kỳ lượt nào; WRITE còn có thể thay đổi trạng thái đơn hàng. Vì vậy, đánh giá chỉ dựa vào reward cuối cùng **không cho biết lỗi xảy ra trước hay sau khi tác động lên cơ sở dữ liệu** đã diễn ra.

τ^2-bench cung cấp tác tử, bộ mô phỏng người dùng, môi trường công cụ và bộ đánh giá kết quả [1]. Trong bản triển khai đang nghiên cứu, mô hình tác tử là `openai/claude-sonnet-5` và mô hình mô phỏng người dùng là `openai/claude-opus-4-8`, được truy cập qua một gateway tương thích OpenAI. Tiền tố `openai/` là cách LiteLLM chọn giao thức gọi API; nó không có nghĩa mô hình nền là của OpenAI. Chỉ có metadata do gateway trả về mới cho phép nói về định tuyến quan sát được; báo cáo không suy đoán backend.

1.2 Câu hỏi nghiên cứu và đóng góp

Báo cáo trả lời ba câu hỏi: (1) lời gọi sai cấu trúc có bị chặn trước khi tác động môi trường không; (2) lỗi provider tạm thời có được thử lại trong giới hạn mà không phát lại WRITE không; (3) khi kết quả sau WRITE không chắc chắn, hệ thống có dừng và giữ bằng chứng thay vì tạo kết quả thành công giả không. Đóng góp gồm một pipeline A1–A3 có thể tắt, dấu vết từng lời gọi và từng attempt provider, cùng bộ chèn lỗi cục bộ để kiểm tra các bất biến. Mục tiêu không phải cải thiện năng lực suy luận, sửa quyết định nghiệp vụ sai của tác tử hay thay đổi bộ đánh giá benchmark.

Chương 2

Nền tảng và phạm vi lỗi

2.1 Luồng thực thi gốc của τ^2-bench

Luồng đơn giản hóa là: mô hình sinh phản hồi → chuyển đổi thành *AssistantMessage* → *Orchestrator* xử lý lượt hội thoại → công cụ môi trường được gọi → kết quả được trả về tác tử → evaluator chấm episode. Phản hồi nhà cung cấp, *AssistantMessage* và lời gọi công cụ là các đối tượng khác nhau. *AssistantMessage* có thể chỉ chứa văn bản, chứa một hoặc nhiều *ToolCall*, hoặc không có nội dung dùng được. **Không phải mọi thất bại trong episode là lỗi của lớp thực thi**: tác tử có thể chọn hành động sai dù lời gọi hợp lệ, người dùng mô phỏng có thể cung cấp thông tin không nhất quán, và tiêu chí đánh giá có thể mơ hồ.

2.2 Phân loại để tránh quy lỗi nhầm

Nhóm	Ý nghĩa và cách xử lý trong nghiên cứu
Sai cấu trúc trước dispatch	JSON không hợp lệ, công cụ không tồn tại, sai requestor, thiếu/thừa đối số, sai kiểu hoặc vi phạm quy tắc miền đã duyệt: A1 từ chối trước khi gọi công cụ.
Lỗi provider/vận hành	Timeout, mã trạng thái được phân loại, mất kết nối, hoặc phản hồi tác tử rỗng: A2 có thể thử lại trước khi tạo *AssistantMessage*; khi hết budget thì không tạo reward giả.
Lỗi công cụ/điều kiện nghiệp vụ	Công cụ Retail tự báo lỗi vì tiền điều kiện không thỏa: giữ hành vi native; không tự động coi là lỗi hạ tầng hay thử lại.
Sai quyết định/đánh giá	Chọn sai hành động, hành vi người dùng mô phỏng khác yêu cầu, hoặc tiêu chí benchmark mơ hồ: cần review trajectory; harness không được dùng ground truth để sửa suy luận.

Bảng 2.1: Các nguồn lỗi có mẫu số và trách nhiệm khác nhau.

Chương 3

Thiết kế và triển khai harness

3.1 Nguyên tắc tích hợp

Harness mặc định tắt. Khi không bind context, đường sinh phản hồi và thực thi gốc vẫn được giữ. Khi bật A1, parser được giới hạn ở đường *LLMAgent* do *Orchestrator* điều khiển; evaluator, bộ kiểm tra ngôn ngữ tự nhiên và các caller khác không đi qua parser này. Mỗi episode có registry snapshot, sidecar, bộ chặn dispatch trùng, môi trường Retail và trace/lifecycle riêng. Kết quả thí nghiệm dùng `max-concurrency=1`. Các điểm nối nằm ở luồng sinh phản hồi, bước xử lý của *Orchestrator* và điều phối episode; không sửa mã công cụ Retail, evaluator hoặc task data.

Provider ⟶ phản hồi LLM ⟶ parser A1 ⟶ registry/schema/miền A1 ⟶ native tool
A2 can thiệp tại biên gọi provider; A3 chỉ can thiệp sau lỗi parser A1 hoặc sau native return và trước khi phát thông báo công cụ thành công.
Kết quả/trace/lifecycle được ghi theo từng episode; evaluator chỉ chấm *SimulationRun* đã hoàn tất.

3.2 A0: baseline native

A0 dùng luồng native của τ^2-bench với model pair và tập nhiệm vụ đã đóng băng. A0 không có A1 validation gate hay A2/A3 sidecar. Provider có thể có cơ chế retry nội bộ của LiteLLM; vì vậy không được diễn giải A0 là "không có bất kỳ retry nào". A0 có kết quả và telemetry hậu kiểm, nhưng không có các bất biến được harness enforce tại thời điểm dispatch.

3.3 A1: kiểm tra và thực thi an toàn

Khi bật A1, đối số thô từ provider đi qua parser; chỉ JSON object mới được chấp nhận. Parser thất bại tạo định danh tổng hợp và một envelope định tuyến, không mang payload lỗi thô. Sau parse, registry xác minh tên công cụ và requestor; tiếp theo là schema nghiêm ngặt (không ép kiểu, không chấp nhận đối số thừa) và các quy tắc miền đã duyệt. Những quy tắc miền này là tập chọn lọc, không phải toàn bộ policy Retail. Một ví dụ được dùng trong fault injection là `modify_pending_order_items`, trong đó ID mặt hàng cũ và mới không được trùng ở cùng vị trí.

Nếu REJECT, A1 không gọi công cụ native, tạo *ToolMessage* có `error=true` và tiếp tục hội thoại khi còn có thể. Nếu ALLOW, dispatcher gọi native đúng một lần và giữ semantics kết quả/lỗi native. Trace JSONL ghi từng pha: `proposed`, `parsed`, `validated`, `dispatch_started`, `dispatch_completed`, `result_emitted`. REJECT không có hai pha dispatch. Lifecycle ghi trạng thái attempt, số event và checksum; lỗi ghi trace là lỗi toàn vẹn nghiên cứu, không được im lặng bỏ qua.

3.4 A2: thử lại ở biên provider

A2 kế thừa A1 và chỉ bật retry cho *agent LLMAgent*, không áp dụng cho mô phỏng người dùng. Một generation có tối đa ba lần gọi provider; nếu thất bại đủ điều kiện, chờ 1 giây rồi 2 giây. Khi A2 bật, retry nội bộ LiteLLM đặt bằng 0 để mỗi attempt của harness tương ứng một lời gọi client. Các lỗi được phân loại theo kiểu ngoại lệ hoặc mã trạng thái có cấu trúc; đường frozen hiện tại gồm phản hồi assistant rỗng, timeout, gián đoạn kết nối, HTTP 502/503 và 429. Không thử lại lỗi domain của công cụ, REJECT của validator hoặc WRITE đã thực thi. Mỗi attempt provider có một dòng JSONL riêng, không được nhầm với *ToolEvent*.

Khi hết ba attempt, hệ thống kết thúc `failed_operational`; **không tạo** *AssistantMessage*, *SimulationRun* hay reward 0 giả. Nếu WRITE đã hoàn tất trước lỗi, attempt bị đánh dấu có thể đã có side effect và cần quyết định thủ công; không tự động phát lại WRITE. Lần chạy A2 final dùng ngân sách resume an toàn theo cấu hình của nó. Các lần thử lỗi được giữ để kiểm toán thay vì ghi đè artifact.

3.5 A3: sửa biểu diễn có giới hạn, kiểm tra đầu ra và cách ly

A3 kế thừa A1+A2, chỉ hoạt động khi được bật rõ ràng. Nó không chạy repair trên lời gọi parse thành công. Sau *ParserFailure* của A1, A3 có thể thử đúng một phép sửa không đoán dữ liệu nghiệp vụ: bỏ hàng rào Markdown quanh JSON, giải mã JSON object bị encode hai lần, hoặc bỏ dấu phẩy thừa trước dấu đóng. Kết quả phải quay lại chính parser A1 rồi đi tiếp qua registry, strict schema và domain validation; JSON bị cắt cụt không được tự bù. **Repair không được bỏ qua A1 validation.**

Với lời gọi WRITE được ALLOW, A3 chụp trạng thái liên quan trước khi gọi native một lần. Ngay sau khi callable trả về, A3 chụp hậu trạng thái, kiểm tra *raw* output theo kiểu trả về đã khai báo, rồi mới đánh giá hậu điều kiện đã viết rõ cho ba công cụ: `cancel_pending_order` (trạng thái và lý do), `return_delivered_order_items` (trạng thái, mặt hàng, phương thức hoàn tiền) và `modify_pending_order_address` (các trường địa chỉ). Chỉ khi kiểm tra qua mới materialize *ToolMessage* thành công. Hash trạng thái là bằng chứng thay đổi, **không tự nó chứng minh thay đổi đúng**. Nếu output không hợp lệ hoặc hậu điều kiện sai sau WRITE, A3 ghi audit và cách ly (*quarantine*); không phát thông báo thành công, không rollback, không tự động replay WRITE.

Tầng	Biên can thiệp	Điều không được suy diễn
A1	Trước native tool	REJECT không đồng nghĩa tác tử đã hoàn thành task.
A2	Trước khi materialize phản hồi provider	Retry provider không phải retry công cụ.
A3	Sau parser lỗi hoặc sau native return, trước thông báo thành công	Kiểm tra output/hậu điều kiện không phải rollback hay sửa mục tiêu nghiệp vụ.

Bảng 3.1: Vị trí và giới hạn của từng tầng harness.

3.6 Artifact và tính kiểm toán

`results.json` chứa các *SimulationRun* hoàn tất và reward của evaluator. JSONL tool trace ghi lời gọi, quyết định ALLOW/REJECT, định danh logic, số dispatch, loại READ/WRITE và hash trạng thái. Tệp `*.lifecycle.json` ràng buộc trạng thái terminal với checksum, số event và sequence cuối. A2 thêm `*.provider-retry.jsonl`; A3 thêm `*.recovery.jsonl` cho quyết định output/postcondition/repair. Mặc định sidecar lưu metadata và hash, không lưu prompt, đối số công cụ thô hay raw provider response. Chỉ xem `results.json` sẽ không thấy các lỗi vận hành chưa tạo *SimulationRun*.

Chương 4

Phương pháp thực nghiệm

4.1 Benchmark tự nhiên và cấu hình đóng băng

Bốn run so sánh dùng cùng một tập 58 task Retail, bốn trial/task (232 tuple dự kiến), root seed 300 và các trial seed 626729, 373753, 361454, 1567. Agent là `openai/claude-sonnet-5`, user simulator là `openai/claude-opus-4-8`; nhiệt độ 0, số bước tối đa 200, `max-errors=10`, concurrency bằng 1. Gateway chỉ là đường truy cập mô hình, không phải một điều kiện thay model. A1 bật validation; A2 bật A1 và provider retry; A3 bật cả A1, A2 và A3. Manifest/config và đường dẫn raw của từng run được nêu ở Phụ lục A. Những root pilot, staging hoặc interrupted cũ không được trộn vào bảng kết quả cuối.

4.2 Định nghĩa metric và mẫu số

Với C là tập episode có *SimulationRun* hợp lệ và P là 232 tuple dự kiến, dùng các định nghĩa:

$$\begin{aligned} \text{Completion} &= |C|/|P|, \\ \text{Pass}^1 &= |\{e \in C : \text{reward}(e) = 1\}|/|C|, \\ \text{DB match} &= |\{e \in C : \text{db_match}(e)\}|/|C|. \end{aligned}$$

Pass^4 là số task có *đủ bốn* trial hoàn tất và cả bốn reward bằng 1, chia cho số task có đủ bốn trial. Nó không chia cho 58 khi task còn thiếu trial; tỷ lệ trên toàn bộ 58 task có thể báo riêng nhưng không gọi chung là Pass^4 ở đây. Các episode A2/A3 chưa hoàn tất vì lỗi vận hành không thuộc mẫu số reward. Bởi vậy **Pass^1 có điều kiện trên completion** và phải luôn đọc kèm Completion.

Với A1–A3, tỷ lệ REJECT→dispatch có tử số là số lời gọi bị REJECT nhưng vẫn có `dispatch_started` hoặc `dispatch_completed` cùng `logical_call_id`; mẫu số là tổng lời gọi REJECT. "Duplicate dispatch" và "duplicate WRITE" yêu cầu *cùng định danh lời gọi logic trong cùng episode* có hơn một lần native execute; không suy từ cùng tên công cụ hoặc cùng đối số. Provider attempts/generation là tổng dòng attempt trong sidecar chia số generation ID duy nhất. Thời lượng trung bình/trung vị lấy từ `duration` của các episode hoàn tất, đơn vị giây.

4.3 Chèn lỗi có kiểm soát

Fault injection (FI) không dùng gateway thật và không đo trí thông minh mô hình. Fixture được biên soạn một lần từ một trajectory A0, sau đó là input độc lập dùng lại cho các cấu hình. Server cục bộ tương thích OpenAI trả phản hồi đã script qua đường LiteLLM thật; lỗi parser/registry/schema được biểu diễn trong phản hồi provider khi có thể, không chèn bằng prompt. Có 14 tình huống lõi C0, P1–P6, D1–D2, R1–R4, S1 chạy ở cả A0–A3: $14 \times 4 = 56$ lần chạy theo cặp tình huống–tầng. Bảy tình huống riêng A3 được chạy với A2 control và A3: $7 \times 2 = 14$ lần. Tổng cộng 70 lần chạy FI; *21* loại tình huống, không phải 22 hay 88 lần chạy. R1 dùng artifact timeout đã sửa ở root FI gapfix. Một FI "PASS" nghĩa hành vi thực thi khớp dự kiến của fault, không có nghĩa reward benchmark bằng 1.

Chương 5

Kết quả và phân tích

5.1 Kết quả benchmark tự nhiên

Các tử/mẫu số dưới đây được tính trên `results.json` của bốn root cuối hợp lệ; số planned là 58×4. Bảng 5.1 phân biệt completion khỏi thành công của những episode đã hoàn tất.

Metric	A0	A1	A2	A3
Completion	232/232 (100,00%)	232/232 (100,00%)	227/232 (97,84%)	221/232 (95,26%)
Pass[1]	186/232 (80,17%)	192/232 (82,76%)	189/227 (83,26%)	177/221 (80,09%)
Pass[4]	37/58 (63,79%)	40/58 (68,97%)	36/54 (66,67%)	31/50 (62,00%)
DB match	186/232 (80,17%)	195/232 (84,05%)	190/227 (83,70%)	180/221 (81,45%)
Reward 0, completed	46	40	38	44
Chưa có benchmark result	0	0	5	11

Bảng 5.1: Kết quả trên các run tự nhiên cuối hợp lệ; Pass[1], Pass[4] và DB match dùng mẫu số episode/task hoàn tất tương ứng.

A1 có Pass[1] cao hơn A0 trong các run này, nhưng A3 không cao hơn A2 và completion giảm ở A2/A3. Vì vậy, **dữ liệu tự nhiên không ủng hộ** phát biểu "A3 tốt hơn A2 tốt hơn A1 tốt hơn A0" về reward. Khác biệt trajectory do provider, người dùng mô phỏng và ngẫu nhiên thực thi vẫn có thể ảnh hưởng kết quả ngay khi task/model/seed được giữ giống nhau. Ở A2, năm tuple chưa có kết quả làm chỉ 54 task đủ bốn trial; ở A3, 11 tuple làm chỉ 50 task đủ bốn trial. Không được lấy 36/58 hay 31/58 rồi gọi đó là Pass[4] theo định nghĩa của báo cáo.

5.2 An toàn thực thi và chi phí vận hành

"Native tool error" ở đây là công cụ thực sự được gọi rồi trả lỗi, thường là điều kiện miền hoặc dữ liệu không hợp lệ; nó không đồng nghĩa lỗi hạ tầng hay crash episode. Agent có thể nhận *ToolMessage* lỗi và tiếp tục. Nó cũng không bao gồm REJECT trước dispatch. Tỷ lệ lỗi native khác nhau giữa các run không tự chứng minh A1/A2/A3 đã sửa lỗi đó: số call và trajectory cũng khác. Trong A1–A3, các trace không ghi nhận vi phạm REJECT→dispatch hoặc duplicate WRITE theo định danh logic. Đây là bằng chứng trên số lần quan sát được, không phải chứng minh bất biến cho mọi input tương lai.

Ở A2, 4 282 lần provider invocation thuộc 4 150 generation; ở A3 là 4 081/3 899. Recovery

Metric	A0	A1	A2	A3
REJECT→dispatch	—	0/3	0/4	0/5
Duplicate dispatch (cùng logical call)	Không enforce	0/1 731	0/1 670	0/1 574
Duplicate WRITE (cùng logical call)	Không enforce	0/355	0/340	0/327
Native tool errors/calls	37/1 737	43/1 731	40/1 670	31/1 574
Provider attempts/generation	—	—	4 282/4 150 (1,032)	4 081/3 899 (1,047)
Thời lượng trung bình (s)	40,18	42,34	47,99	53,03
Thời lượng trung vị (s)	36,84	39,86	43,71	40,54

Bảng 5.2: Quan sát thực thi trên run tự nhiên. A0 không có A1 trace để enforce các bất biến tương ứng; — không có nghĩa bằng 0.

trong *cùng generation có retry* là 0/66 (A2) và 5/93 (A3). Sự khác biệt này không đủ để quy tăng recovery cho A3: A3 không sở hữu retry provider và các run tự nhiên có điều kiện vận hành khác nhau. Mean duration tăng theo tầng nhưng median A3 (40,54 s) thấp hơn median A2 (43,71 s); không nên gọi chênh lệch wall-clock là overhead thuần của harness vì nó bao gồm thời gian gateway và các trajectory khác nhau.

5.3 Bằng chứng riêng của A3

Lọc `*.recovery.jsonl` theo 221 episode có lifecycle `closed` và *SimulationRun* hợp lệ cho thấy 1 543 event `output_valid`, trong đó 162 lượt hậu điều kiện WRITE áp dụng và đều đạt; không ghi nhận repair hay quarantine tự nhiên trong tập episode hoàn tất này. **Số 0 đó không chứng minh A3 vô ích**: khi fault không xảy ra, khả năng repair/quarantine phải được kiểm tra bằng FI. Không trộn event của các attempt thất bại vào mẫu số episode hoàn tất.

5.4 Kết quả chèn lỗi và phát biểu tăng thêm

Ví dụ cụ thể: với JSON được bọc trong Markdown fence, A3 thực hiện một lần repair, parser và validator A1 chạy lại, rồi native dispatch một lần. Với output không hợp lệ sau WRITE, FI A3 ghi một WRITE đã dispatch và một quarantine, không phát *ToolMessage* thành công và không replay. Với provider exhaustion sau WRITE, A2/A3 giữ attempt lỗi làm chứng cứ vận hành, không tự biến nó thành benchmark result. Các tình huống này chỉ chứng minh **hành vi tại boundary được thử**; không chứng minh gateway doanh nghiệp hay mọi công cụ Retail ngoài fixture đều có cùng hành vi.

5.5 Phân tích reward 0 và lỗi vận hành

Review thủ công các episode reward 0 của bốn run cho thấy các nhãn chính như Bảng 5.4. Nhãn được gán bằng task/evaluation criteria, toàn bộ trajectory, kết quả công cụ và trace; không suy từ reward 0 hoặc `ToolMessage.error=true` một cách đơn lẻ. **Reward 0 không**

Nhóm tình huống	A0/A1/A2	Khả năng được kiểm chứng ở A3
C0: đường sạch	Cả ba giữ luồng hợp lệ; A1/A2 khớp thông báo công cụ native trong fixture.	Đường sạch A3 không phát sinh repair.
P1–P6, D1	A1 chặn parse, registry, schema, domain trước native; A2 kế thừa.	A3 giữ A1 containment, không tự sửa JSON tùy ý.
D2: lỗi native có kiểm soát	Công cụ native trả điều kiện miền; không bị A1/A2 thử lại.	A3 vẫn giữ lỗi native, không nhận nhầm là hạ tầng.
R1–R3	A2 phục hồi fault timeout, 503 và reasoning-only trong FI.	A3 kế thừa retry của A2, không thêm retry tool.
R4, S1	A2 chặn exhaustion trước WRITE và không replay WRITE đã hoàn tất.	A3 giữ chính sách không replay và bằng chứng terminal.
7 fault riêng A3	A2 control hiện diện cho cả bảy fault.	Ba lỗi biểu diễn được sửa rồi quay lại A1; truncated JSON không sửa; repair nhưng sai schema vẫn REJECT; hai lỗi sau WRITE bị cách ly, WRITE đúng một lần.

Bảng 5.3: Tóm tắt FI theo khả năng thực thi, không phải reward benchmark.

tự động là lỗi harness.

Nhãn chính của completed reward 0	A0	A1	A2	A3
Sai suy luận/chọn hành động của model	24	27	24	25
Người dùng mô phỏng lệch yêu cầu	13	11	9	14
Mơ hồ benchmark/evaluator	8	1	4	5
Không đủ bằng chứng quy nguyên nhân	1	1	1	0
Tổng	46	40	38	44

Bảng 5.4: Phân loại nguyên nhân chính của episode đã hoàn tất nhưng reward 0; không bao gồm operationally unresolved.

Đa số nhãn trong bảng không phải lỗi harness cần hoặc được phép chặn. Đặc biệt, dùng ground truth của evaluator để ép hành động agent sẽ làm hỏng tính độc lập của benchmark. Năm tuple A2 và 11 tuple A3 chưa có benchmark result được ghi nhận ở operational accounting, không nhập vào bốn hàng reward 0. Một số phản hồi provider có `content` và `tool_calls` rỗng nhưng `reasoning_content` tồn tại. Đây là quan sát cấu trúc ở phía client, không đủ chứng minh nguyên nhân tại gateway/backend. Chính sách hiện tại không chuyển reasoning thành lời gọi công cụ hay văn bản trả lời.

Chương 6

Giới hạn, khả năng tái lập và hướng tiếp theo

6.1 Giới hạn suy luận từ kết quả

6.1.1 Benchmark tự nhiên chưa tạo đủ áp lực cho lớp thực thi

Thí nghiệm chỉ bao phủ **58 task Retail chọn lọc, một cặp mô hình, một gateway và concurrency bằng 1**. Số REJECT tự nhiên ở A1, A2, A3 lần lượt chỉ là 3, 4, 5; trong 221 episode A3 hoàn tất không quan sát thấy repair hoặc quarantine. Điều này không nói các cơ chế ấy không cần thiết, mà cho thấy **benchmark tự nhiên chưa kích hoạt đủ các trường hợp hiếm nhưng quan trọng** để tự nó đánh giá toàn diện lớp thực thi. Các run cũng không phải phản thực nghiệm trên cùng một trajectory: A2/A3 thiếu episode, còn hành vi model và gateway có thể khác giữa các lần chạy.

6.1.2 Fault injection có kiểm soát chưa phản ánh đầy đủ lỗi thực tế

Bộ FI giữ cố định fixture và vị trí lỗi, nhờ đó có thể quy kết phản ứng tại từng boundary. Tuy nhiên, **21 loại tình huống FI không phải phân bố lỗi của lưu lượng thật**. Nó chưa tái tạo đầy đủ ngữ cảnh dài, lỗi liên tiếp có tương quan, biến động gateway/backend, concurrency hoặc side effect của hệ thống bên ngoài. FI vì vậy chứng minh khả năng xử lý đối với fault đã đăng ký, *không* cho phép ngoại suy trực tiếp sang tỷ lệ lỗi hay mức cải thiện reward trong triển khai thực tế.

6.1.3 Bộ metric chưa bao quát toàn bộ độ tin cậy vận hành

Pass^1, Pass^4 và DB match mô tả kết quả benchmark; REJECT→ dispatch, duplicate WRITE và trace integrity đo một số bất biến an toàn; provider attempts và duration đo một phần chi phí. Nhưng **chưa có một metric duy nhất tổng hợp chính xác mức độ an toàn, khả dụng và chi phí** cho mọi trạng thái. Đặc biệt, completion thấp hơn có thể phản ánh chính sách dừng an toàn thay vì mô hình làm task kém hơn; ngược lại, reward cao trên tập completed có thể che các tuple chưa hoàn tất. Việc cần review thủ công, thời gian tới khi khôi phục, chi phí provider và mức độ hại của một side effect chưa được lượng hóa tương

xứng trong các bảng headline.

6.1.4 A3 là phục hồi có kiểm soát, chưa phải phục hồi tự chủ

A3 chỉ sửa **ba dạng lỗi biểu diễn** đã xác định và kiểm hậu điều kiện cho **ba WRITE Retail** có quy tắc tường minh. Với output mơ hồ hoặc hậu điều kiện sai sau WRITE, A3 cách ly và giữ bằng chứng; nó không rollback hay tự động replay. Hash trạng thái đổi không đồng nghĩa kết quả nghiệp vụ đúng. **Cách ly/manual audit là giới hạn an toàn có chủ đích**, nhưng đồng thời tạo gánh nặng vận hành và có thể giảm completion. Đây chưa phải *autonomous recovery* cho các công cụ có side effect bất kỳ.

6.2 Hướng phát triển có điều kiện

6.2.1 Đánh giá nhiều mô hình và nhiều mức năng lực

Chạy cùng protocol trên các mô hình mạnh, trung bình và yếu để xem lớp thực thi chịu áp lực khác nhau ra sao khi tần suất gọi sai tool, sai schema hoặc trả phản hồi không sử dụng được thay đổi. Cần giữ cố định task, seed, evaluator và quy tắc tính metric; báo cả completion và lỗi vận hành bên cạnh reward, thay vì chỉ xếp hạng model theo Pass[1]. Có thể mở rộng thêm domain và concurrency sau khi các bất biến episode isolation còn đúng.

6.2.2 Mô hình hóa fault sát với run tự nhiên hơn

Thu thập mẫu hình lỗi từ natural runs và phân tầng theo task, turn, độ dài ngữ cảnh, loại công cụ, vị trí trước/sau WRITE và metadata provider. Từ đó xây các fault có tần suất, chuỗi lỗi và ngữ cảnh gần thực tế hơn, nhưng vẫn có fixture đóng băng và kiểm soát được injection point. Không dùng raw prompt, dữ liệu khách hàng hoặc một nhận định chưa xác minh về gateway làm đầu vào công khai. Với phản hồi assistant rỗng/chỉ có reasoning, cần đối chiếu correlation ID cùng nhóm gateway/provider để phân biệt lỗi backend, gateway transformation, giới hạn dịch vụ hay client.

6.2.3 Tiến từ containment tới phục hồi tự động có chứng minh

Lộ trình là **phát hiện/cách ly → deterministic repair → safe automatic recovery**, không nhảy thẳng sang phát lại WRITE. Một cơ chế tự động mới chỉ nên bật khi có hợp đồng output và hậu điều kiện xác định, state snapshot đáng tin, ràng buộc idempotency hoặc bằng chứng không thể tạo side effect lặp; nếu thiếu một điều kiện, tiếp tục quarantine/manual review. Mở rộng hậu điều kiện từng công cụ phải dựa vào policy công khai và kiểm thử được, **không dùng ground truth evaluator** để ép hành động của agent. Song song, bổ sung metric cho số ca cần người duyệt, thời gian khôi phục, false reject và chi phí vận hành để đánh giá được lợi ích cùng cái giá của phục hồi.

6.3 Khả năng tái lập và đối chiếu artifact

Từng bảng số liệu có nguồn ở Phụ lục A. Để tái lập: (1) đọc manifest/ledger xác định tập task và seed; (2) đọc `results.json` lấy reward, DB match và duration; (3) join tool trace theo episode/`logical_call_id` để đo REJECT, dispatch và WRITE; (4) đọc provider/recovery sidecar cho attempt và postcondition; (5) kiểm checksum/count/sequence trong lifecycle trước khi kết luận. Không ghi đè raw artifact của bất kỳ stage nào khi tạo phân tích. Các script và hướng dẫn thao tác của repo được mô tả thêm trong `README.md` và tài liệu `analysis/METRIC_CATALOG_A0_A3_HOW_TO_CALCULATE.md`.

Chương 7

Kết luận

Báo cáo triển khai **Lớp Thực Thi Tool Đáng Tin Cậy** với ba tầng cho τ^2-bench Retail mà vẫn giữ đường native khi tắt: A1 chặn lời gọi sai trước dispatch, A2 thử lại lỗi provider có giới hạn và ghi lineage, A3 sửa biểu diễn không suy diễn hoặc cách ly kết quả sau WRITE không an toàn. Benchmark tự nhiên cho thấy tỷ lệ hoàn tất và reward không tăng đơn điệu từ A0 đến A3; không thể dùng chúng để khẳng định tác tử suy luận tốt hơn. **Giá trị đã được kiểm chứng nằm ở đúng biên thực thi**: REJECT không gọi native, WRITE không bị tự phát lại, lỗi provider không bị gán reward giả, và lỗi sau WRITE có trace để kiểm toán. Thí nghiệm FI cục bộ cung cấp bằng chứng cơ chế cho các phát biểu đó; điều tra nguồn gốc phản hồi rỗng và mở rộng đánh giá vẫn là công việc tiếp theo.

Phụ lục A

Nguồn số liệu và phiên bản run

Các đường dẫn sau tính từ thư mục `data/simulations/`. Không dùng root pilot/staging hoặc run historical interrupted thay thế. Danh sách 58 task đã khóa là: 8, 9, 14, 16, 17, 18, 20, 21, 22, 23, 24, 25, 28, 35, 37, 38, 39, 40, 44, 46, 48, 50, 52, 53, 55, 58, 60, 61, 62, 63, 64, 68, 70, 71, 73, 74, 75, 76, 77, 78, 79, 80, 82, 84, 86, 87, 88, 89, 91, 93, 95, 96, 101, 103, 104, 107, 110, 113.

- A0: `20_a0_native_baseline/a0-main/retail-stratified-58-v1/results/results.json`; ledger, telemetry và phân tích ở cùng root.
- A1: `30_a1_validation/a1-main/retail-stratified-58-v1-4trials-v1/results.json`; tool trace/lifecycle tại thư mục `traces/` của root này.
- A2: `40_a2_validation_retry/a2-main/retail-stratified-58-v1-4trials-v4-structural-obse results/results.json`; provider sidecar và operational accounting ở `traces/`.
- A3: `50_a3_validation_retry_recovery/a3-main/retail-stratified-58-v1-4trials-v3-per-lo results/results.json`; manifest, ledger, tool trace, provider/recovery sidecar và accounting ở cùng root.
- FI coverage: `60_fault_injection/analysis/fi_v1_a0_a3_coverage_v1/fault_coverage_reconciliation.md`; FI A3 pilot: `60_fault_injection/fi-v1-a3-pilot-v1/analysis/a3_fi_pilot_report.md`.

Phụ lục B

Định nghĩa đối chiếu nhanh các trường

Trong `results.json`, mỗi phần tử của `simulations` có `task_id`, `trial`, `duration` và `reward_info.reward`; DB match lấy từ `reward_info.db_check.db_match`. Trong tool trace JSONL, các trường quan trọng là `phase`, `origin`, `decision`, `logical_call_id`, `dispatch_count`, `dispatch_status` và `native_tool_type`. A2 sidecar dùng `generation_id`, `provider_attempt`, `failure_code` và `outcome`. A3 recovery sidecar dùng `disposition`, `repair_attempt_count`, `output_validation` và `postcondition`. Mỗi thống kê theo attempt phải được phân biệt với thống kê theo episode có kết quả; không cộng gộp hai đơn vị quan sát.

Tài liệu tham khảo

[1] Nhóm phát triển τ^2-bench, *τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment*, arXiv:2506.07982, 2025. https://arxiv.org/abs/2506.07982.

[2] Nhóm phát triển τ-bench, *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*, arXiv:2406.12045, 2024. https://arxiv.org/abs/2406.12045.

[3] Mã nguồn và artifact thực nghiệm của báo cáo, https://github.com/muoimeo/tau2-tool-execution-harness, phiên bản cơ sở τ^2-bench v0.1.3 cùng các thư mục `src/tau2/harness/`, `data/simulations/` và `analysis/`.